Activity 8.23

In your chosen programming language, write a program to declare and populate the array **MyTable**, as shown in Figure 8.11, using a nested **FOR** loop.

8.3 File handling

8.3.1 Purpose of storing data in a file

Computer programs store data that will be required again in a **file**. While any data stored in RAM will be lost when the computer is switched off, when data is saved to a file it is stored permanently. Data stored in a file can thus be accessed by the same program at a later date or accessed by another program. Data stored in a file can also be sent to be used on other computer(s). The storage of data in files is one of the most used features of programming.

8.3.2 Using files

Every file is identified by its filename. In this section, we are going to look at how to read and write a line of text or a single item of data to a file.

Here are examples of writing a line of text to a file and reading the line of text back from the file. The pseudocode algorithm has comments to explain each stage of the task.

pseudocode

```
DECLARE TextLine : STRING  // variables are declared as normal
DECLARE MyFile : STRING
MyFile ← "MyText.txt"
// writing the line of text to the file
OPEN MyFile FOR WRITE  // opens file for writing
    OUTPUT "Please enter a line of text"
    INPUT TextLine
    WRITEFILE, TextLine  // writes a line of text to the file
CLOSEFILE(MyFile)  // closes the file
// reading the line of text from the file
OUTPUT "The file contains this line of text:"
OPEN MyFile FOR READ  // opens file for reading
    READFILE, TextLine  // reads a line of text from the file
    OUTPUT TextLine
CLOSEFILE(MyFile)  // closes the file
```

Python

```
# writing to and reading a line of text from a file
MyFile = open ("MyText.txt","w")
TextLine = input("Please enter a line of text ")
MyFile.write(TextLine)
Myfile.close()
print("The file contains this line of text")
MyFile = open ("MyText.txt","r")
TextLine = MyFile.read()
print(TextLine)
Myfile.close()
```

Visual Basic

```
'writing to and reading from a text file
Imports System.IO
Module Module1
    Sub Main()
        Dim textLn As String
        Dim objMyFileWrite As StreamWriter
        Dim objMyFileRead As StreamReader
        objMyFileWrite = New StreamWriter("textFile.txt")
        Console.Write("Please enter a line of text  ")
        textLn = Console.ReadLine()
        objMyFileWrite.WriteLine(textLn)
        objMyFileWrite.Close()
        Console.WriteLine("The line of text is  ")
        objMyFileRead = New StreamReader("textFile.txt")
        textLn = objMyFileRead.ReadLine
        Console.WriteLine(textLn)
        objMyFileRead.Close()
        Console.ReadLine()
    End Sub
End Module
```

Java

```
//writing to and reading from a text file, Note: the try and catch commands implement
// something called exception handling - this concept goes beyond IGCSE level
import java.util.Scanner;
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
class TextFile {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in);
        String textLn;
       try {
            FileWriter myFileWriter = new FileWriter("textFile.txt", false);
            PrintWriter myPrintWriter = new PrintWriter(myFileWriter);
            System.out.println("Please enter a line of text  ");
            textLn = myObj.next();
            myPrintWriter.printf("%s" + "%n", textLn);
            myPrintWriter.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
        try {
            FileReader myFileReader  = new FileReader("textFile.txt");
            BufferedReader myBufferReader  = new BufferedReader(myFileReader);
            textLn = myBufferReader.readLine();
            System.out.println(textLn);
            myFileReader.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Activity 8.24

Use the program written in your chosen programming language to write a line of text to a file and read it back from the file. Use the comments in the pseudocode to help you write comments to explain how the file handling works.

Activity 8.25

Using pseudocode write an algorithm to copy a line of text from one text file to another text file.

In this chapter, you have learnt about:

- ✔ declare, use and identify appropriate data types for variables and constants
- ✔ understand and use input and outputs
- ✔ understand and use the programming concepts: sequence, selection, iteration, totalling, counting and string handling
- ✔ understand and use nested statements
- ✔ understand and use arithmetic, logical and Boolean operators
- ✔ understand and use procedures, functions and library routines
- ✔ write maintainable programs
- ✔ declare, use, identify appropriate data types for, read and write arrays with one and two dimensions
- ✔ store data in a file and retrieve data from a file.

Key terms used throughout this chapter

variable – a named data store that contains a value that may change during the execution of a program

constant – a named data store that contains a value that does not change during the execution of a program

declare – define the value and data type of a variable or constant

integer – a positive or negative whole number that can be used with mathematical operators

real number – a positive or negative number with a fractional part; Real numbers can be used with mathematical operators

char – a variable or constant that is a single character

string – a variable or constant that is several characters in length. Strings vary in length and may even have no characters (an empty string); the characters can be letters and/or digits and/or any other printable symbol

sequence – the order in which the steps in a program are executed

selection – allowing the selection of different paths through the steps of a program

iteration – a section of programming code that can be repeated under certain conditions

counting – keeping track of the number of times an action is performed

totalling – keeping a total that values are added to

operator – a special character or word in a programming language that identifies an action to be performed

arithmetic operator – an operator that is used to perform calculations

logical operator – an operator that is used to decide the path to take through a program if the expression formed is true or false

Boolean operator – an operator that is used with logical operators to form more complex expressions

nesting – the inclusion of one type of code construct inside another

procedure – a set of programming statements grouped together under a single name that can be called to perform a task in a program, rather than including a copy of the code every time the task is performed

function – a set of programming statements grouped together under a single name which can be called to perform a task in a program, rather than including a copy of the code every time; just like a procedure except a function will return a value back to the main program

parameters – the variables in a procedure or function declaration that store the values of the arguments passed from the main program to a procedure or function

MOD – an arithmetic operator that returns the remainder of a division; different languages use different symbols for this operation

DIV – an arithmetic operator that returns the quotient (whole number part) of a division; different languages use different symbols for this operation

ROUND – a library routine that rounds a value to a given number of decimal places

RANDOM – a library routine that generates a random number

array – a data structure containing several elements of the same data type; these elements can be accessed using the same identifier name

index – identifies the position of an element in an array

file – a collection of data stored by a computer program to be used again